

Balance is one of the fundamental laws of the universe. It applies to everything, either in the fact that it is balanced or in the fact that it's in a state of flux trying to retrieve it.



How to check if a BST is balanced?

Balance Factor of a node = Height of left subtree - Height of right subtree

0/1/-1: Balanced

2/-2: Unbalanced

Animation: <https://youtu.be/NHNDNoA7Dzw>

Insertions in a Balanced Binary Search Tree (AVL Tree)

1. Insert the node the same way as you do for a BST.
2. If it created unbalance, check the deepest unbalanced node and perform suitable rotation.

LL (Imbalance coming from left of left subtree): Do a “Right” rotation.

Animation: <https://youtu.be/J8SODH7vGuw>

Insertions in a Balanced Binary Search Tree (AVL Tree)

1. Insert the node the same way as you do for a BST.
2. If it created unbalance, check the deepest unbalanced node and perform suitable rotation.

RR (Imbalance coming from right of right subtree): Do a “Left” rotation.

Animation: <https://www.youtube.com/watch?v=97SeN296Z50>

Insertions in a Balanced Binary Search Tree (AVL Tree)

1. Insert the node the same way as you do for a BST.
2. If it created unbalance, check the deepest unbalanced node and perform suitable rotation.

LR (Imbalance coming from right of left subtree): Do a “Right” rotation on slightly unbalanced child (BF: 1) of unbalanced (BF: -2) node and then do a “Left” rotation on the unbalanced (BF: -2) node

Animation: https://youtu.be/pZgOy_aMAPI

Insertions in a Balanced Binary Search Tree (AVL Tree)

1. Insert the node the same way as you do for a BST.
2. If it created unbalance, check the deepest unbalanced node and perform suitable rotation.

RL (Imbalance coming from left of right subtree): Do a “Left” rotation on slightly unbalanced child (BF: -1) of unbalanced (BF: 2) node and then do a “Right” rotation on the unbalanced (BF: 2) node

Animation: <https://youtu.be/uHJFdOVi9oM>

Deletion in a Balanced Binary Search Tree (AVL Tree)

1. Delete the node the same way as you do for a BST.
2. If it created unbalance, check the deepest unbalanced node and perform suitable rotation.
3. Use any of the case (LL/RR/LR/RL) suitably.

Above might unbalance a parent/grandparent, so you might have to balance recursively.

Animation:

<https://youtu.be/v7WAlh9-IMs>

<https://youtu.be/OIJAFUT16Nw>

Complexities: Insertion, Search, Deletion

Method	Insertion	Search	Deletion
Unordered Array	$O(1)$	$O(n)$	$O(n)$
Sorted Array	$O(n)$	$O(\log n)$	$O(n)$
BST	$O(n)$	$O(n)$	$O(n)$
AVL Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$